Data Preparation

Date	24 September 2026
Team ID	
Project Name	EV Market Analytics Dashboard
Maximum Marks	5 Marks

Data preparation refers to the process of cleaning, standardizing, transforming, and organizing data so that it is suitable for analysis and reporting. In a Snowflake project, data preparation involves working with the ingested RAW data, identifying data quality issues, creating STAGING tables, applying required transformations, and preparing reliable data for the CURATED and MART layers.

Activity 1: Identifying and Handling Null Values

Identify the NULL or missing values present in the Snowflake dataset, analyze the affected columns, determine the appropriate approach for handling them based on the data and business requirements, apply the required query, and validate the cleaned data to ensure it is ready for further analysis and visualization.

Code:

```
USE DATABASE EV_DB;
USE WAREHOUSE EV_WH;

-- Step 1: identify NULLs in every column of the RAW table
SELECT COUNT(*)                                    AS TOTAL_ROWS,
       COUNT_IF(MANUFACTURER IS NULL)              AS NULL_MANUFACTURER,
       COUNT_IF(VEHICLE_CATEGORY IS NULL)          AS NULL_CATEGORY,
       COUNT_IF(BATTERY_TYPE IS NULL)              AS NULL_BATTERY_TYPE,
       COUNT_IF(BATTERY_CAPACITY_KWH IS NULL)      AS NULL_CAPACITY,
       COUNT_IF(DRIVING_RANGE_KM IS NULL)          AS NULL_RANGE,
       COUNT_IF(PRICE_USD IS NULL)                 AS NULL_PRICE,
       COUNT_IF(ANNUAL_SALES IS NULL)              AS NULL_SALES,
       COUNT_IF(SALES_YEAR IS NULL)                AS NULL_YEAR,
       COUNT_IF(MANUFACTURING_COUNTRY IS NULL)     AS NULL_COUNTRY,
       COUNT_IF(CUSTOMER_RATING IS NULL)           AS NULL_RATING
FROM EV_DB.RAW.EV_DATA_RAW;

-- Step 2: handle NULLs while creating the STAGING table
--   key columns (manufacturer, category, year) missing -> record removed
--   text columns missing -> 'UNKNOWN'; missing sales -> 0
--   numeric measures (range, capacity, rating) kept as NULL so AVG() ignores them
CREATE OR REPLACE TABLE EV_DB.STAGING.EV_DATA_STG AS
SELECT VEHICLE_ID, MANUFACTURER, MODEL, VEHICLE_CATEGORY,
       COALESCE(BATTERY_TYPE, 'UNKNOWN')           AS BATTERY_TYPE,
       BATTERY_CAPACITY_KWH, DRIVING_RANGE_KM, PRICE_USD,
       COALESCE(ANNUAL_SALES, 0)                   AS ANNUAL_SALES,
       SALES_YEAR,
       COALESCE(MANUFACTURING_COUNTRY, 'UNKNOWN')  AS MANUFACTURING_COUNTRY,
       CUSTOMER_RATING
FROM EV_DB.RAW.EV_DATA_RAW
WHERE MANUFACTURER IS NOT NULL
  AND VEHICLE_CATEGORY IS NOT NULL
  AND SALES_YEAR IS NOT NULL;

-- Step 3: validate - key columns must now have zero NULLs
SELECT COUNT(*) AS STG_ROWS,
       COUNT_IF(MANUFACTURER IS NULL OR VEHICLE_CATEGORY IS NULL
                OR SALES_YEAR IS NULL OR BATTERY_TYPE IS NULL) AS REMAINING_KEY_NULLS
FROM EV_DB.STAGING.EV_DATA_STG;
```

Output:

[Add output screenshot]

Activity 2: Duplicate Record Handling

Identify and analyze duplicate records in the Snowflake dataset, determine whether the duplicates are genuine or repeated entries, define the appropriate criteria for identifying duplicates, remove or retain records based on the project requirements, and validate the dataset after treatment to ensure that duplicate records do not affect the accuracy of further analysis and visualizations.

Code:

```
-- Step 1: identify duplicates (same manufacturer, model and year)
SELECT MANUFACTURER, MODEL, SALES_YEAR, COUNT(*) AS DUPLICATE_COUNT
FROM EV_DB.STAGING.EV_DATA_STG
GROUP BY MANUFACTURER, MODEL, SALES_YEAR
HAVING COUNT(*) > 1
ORDER BY DUPLICATE_COUNT DESC;

-- Step 2: keep one record per manufacturer + model + year
CREATE OR REPLACE TABLE EV_DB.STAGING.EV_DATA_STG AS
SELECT *
FROM EV_DB.STAGING.EV_DATA_STG
QUALIFY ROW_NUMBER() OVER (
          PARTITION BY MANUFACTURER, MODEL, SALES_YEAR
          ORDER BY VEHICLE_ID) = 1;

-- Step 3: validate - no duplicates should remain
SELECT COUNT(*) AS ROWS_AFTER_DEDUP,
       COUNT(*) - COUNT(DISTINCT MANUFACTURER || '|' || MODEL || '|' || SALES_YEAR)
         AS REMAINING_DUPLICATES
FROM EV_DB.STAGING.EV_DATA_STG;
```

Output:

[Add output screenshot]

Activity 3: Data Cleaning and Standardization

Identify and correct inconsistencies, invalid entries, formatting issues, and data quality problems in the Snowflake dataset, standardize values and formats across relevant columns, apply the required SQL-based cleaning and transformation steps, and validate the cleaned and standardized dataset to ensure consistency, accuracy, and readiness for further analysis and visualization.

Code:

```
-- Step 1: trim and standardise text columns
UPDATE EV_DB.STAGING.EV_DATA_STG
SET MANUFACTURER         = INITCAP(TRIM(MANUFACTURER)),
    MODEL                = TRIM(MODEL),
    VEHICLE_CATEGORY     = UPPER(TRIM(VEHICLE_CATEGORY)),
    MANUFACTURING_COUNTRY = INITCAP(TRIM(MANUFACTURING_COUNTRY));

-- Step 2: standardise battery type names to one consistent value each
UPDATE EV_DB.STAGING.EV_DATA_STG
SET BATTERY_TYPE = CASE
    WHEN UPPER(TRIM(BATTERY_TYPE)) IN ('LFP', 'LITHIUM IRON PHOSPHATE') THEN 'LFP'
    WHEN UPPER(TRIM(BATTERY_TYPE)) IN ('NMC', 'NICKEL MANGANESE COBALT') THEN 'NMC'
    WHEN UPPER(TRIM(BATTERY_TYPE)) IN ('NCA', 'NICKEL COBALT ALUMINUM') THEN 'NCA'
    WHEN UPPER(TRIM(BATTERY_TYPE)) IN ('SODIUM-ION', 'SODIUM ION') THEN 'SODIUM-ION'
    WHEN UPPER(TRIM(BATTERY_TYPE)) IN ('SOLID-STATE', 'SOLID STATE') THEN 'SOLID-STATE'
    ELSE UPPER(TRIM(BATTERY_TYPE)) END;

-- Step 3: remove invalid records (rating scale assumed 0-5)
DELETE FROM EV_DB.STAGING.EV_DATA_STG
WHERE ANNUAL_SALES < 0
   OR DRIVING_RANGE_KM <= 0
   OR BATTERY_CAPACITY_KWH <= 0
   OR PRICE_USD <= 0
   OR CUSTOMER_RATING < 0 OR CUSTOMER_RATING > 5;

-- Step 4: validate - distinct values should now be clean and consistent
SELECT DISTINCT BATTERY_TYPE FROM EV_DB.STAGING.EV_DATA_STG ORDER BY 1;
```

```
SELECT DISTINCT VEHICLE_CATEGORY FROM EV_DB.STAGING.EV_DATA_STG ORDER BY 1;
SELECT COUNT(*) AS CLEAN_ROWS FROM EV_DB.STAGING.EV_DATA_STG;
```

Output:

[Add output screenshot]

Activity 4: Data Transformation and Validation

Apply the required data transformations to prepare the cleaned data for analysis and modelling. Create necessary derived fields, apply business rules, filter or transform records where required, and validate the prepared data and other relevant quality checks before using it for the CURATED and MART layers.

Code:

```
-- Step 1: CURATED table with derived fields and business rules
CREATE OR REPLACE TABLE EV_DB.CURATED.EV_DATA_CLEAN AS
SELECT s.*,
       ROUND(DRIVING_RANGE_KM / NULLIF(BATTERY_CAPACITY_KWH, 0), 2) AS RANGE_PER_KWH,
       ANNUAL_SALES * PRICE_USD                                      AS ESTIMATED_REVENUE_USD,
       CASE WHEN CUSTOMER_RATING >= 4.5 THEN 'Excellent'
            WHEN CUSTOMER_RATING >= 4.0 THEN 'Good'
            WHEN CUSTOMER_RATING >= 3.0 THEN 'Average'
            WHEN CUSTOMER_RATING IS NULL THEN 'Not Rated'
            ELSE 'Low' END                                           AS RATING_BAND
FROM EV_DB.STAGING.EV_DATA_STG s;

-- Step 2: MART views used by the Streamlit dashboard
CREATE OR REPLACE VIEW EV_DB.MART.VW_CATEGORY_SALES AS
SELECT SALES_YEAR, VEHICLE_CATEGORY,
       SUM(ANNUAL_SALES)              AS TOTAL_SALES,
       ROUND(AVG(DRIVING_RANGE_KM),1) AS AVG_RANGE_KM,
       ROUND(AVG(CUSTOMER_RATING),2)  AS AVG_RATING
FROM EV_DB.CURATED.EV_DATA_CLEAN
GROUP BY SALES_YEAR, VEHICLE_CATEGORY;

CREATE OR REPLACE VIEW EV_DB.MART.VW_MANUFACTURER_SHARE AS
SELECT MANUFACTURER,
       SUM(ANNUAL_SALES) AS TOTAL_SALES,
       ROUND(100 * SUM(ANNUAL_SALES) / SUM(SUM(ANNUAL_SALES)) OVER (), 2) AS MARKET_SHARE_PCT,
       ROUND(AVG(CUSTOMER_RATING), 2) AS AVG_RATING
FROM EV_DB.CURATED.EV_DATA_CLEAN
GROUP BY MANUFACTURER;

CREATE OR REPLACE VIEW EV_DB.MART.VW_BATTERY_SUMMARY AS
SELECT BATTERY_TYPE, SALES_YEAR,
       COUNT(*)                          AS MODEL_COUNT,
       SUM(ANNUAL_SALES)                 AS TOTAL_SALES,
       ROUND(AVG(DRIVING_RANGE_KM), 1)   AS AVG_RANGE_KM,
       ROUND(AVG(BATTERY_CAPACITY_KWH), 1) AS AVG_CAPACITY_KWH
FROM EV_DB.CURATED.EV_DATA_CLEAN
GROUP BY BATTERY_TYPE, SALES_YEAR;

-- Step 3: validation checks before using the CURATED and MART layers
SELECT 'RAW' AS LAYER, COUNT(*) AS ROW_COUNT FROM EV_DB.RAW.EV_DATA_RAW
UNION ALL SELECT 'STAGING', COUNT(*) FROM EV_DB.STAGING.EV_DATA_STG
UNION ALL SELECT 'CURATED', COUNT(*) FROM EV_DB.CURATED.EV_DATA_CLEAN;

-- total sales in the MART view must equal total sales in the CURATED table
SELECT (SELECT SUM(ANNUAL_SALES) FROM EV_DB.CURATED.EV_DATA_CLEAN) AS CURATED_SALES,
       (SELECT SUM(TOTAL_SALES) FROM EV_DB.MART.VW_CATEGORY_SALES) AS MART_SALES;
```

Output:

[Add output screenshot]